APSC 143: Introduction to Computer Programming for Engineers

Assignment #3 - Arrays and Strings

Fall 2026



Figure 1: The Limestone 5K. Original image source: KRRA.org

1 Introduction

In this assignment, you will have to write functions in C that create and manipulate arrays of numerical and character data (e.g., arrays and strings). These functions will be combined together to present numerical results in human-readable formats.

Specifically, you will calculate a pace table, which is the time it takes to run different distances at various running paces.

To achieve this, you will implement at least four different functions that calculate:

1. A one-dimensional array of N values evenly spaced between starting and ending values.
2. A human readable time string of hours, minutes, and seconds from a given fractional number of minutes.

 ©2026 Queen's University

3. A two dimensional array of the time required to run given distances at given running paces stored as the fractional number of minutes. Each column of the table will correspond to a distance, and each row to a pace.
4. A two dimensional array of the time required to run given distances at given running paces stored as human readable strings of integer hours, minutes, and seconds. Each column of the table will correspond to a distance, and each row to a pace.

The specific programming instructions for the assignment are provided in Section 2. The background on pace tables is in Section 3. Details on how to submit your work are provided in Section 4

2 Instructions

In this assignment, you will create C functions to create pace tables. Specifically, you must implement the following four functions given the compiler macros `ROWS`, `COLUMNS`, and `LENGTH`:

1. `bool linspace(double firstValue, double lastValue, int numValues, double outputArray[])`
   Calculates and returns a one-dimensional double array with `numValues` where the first value is equal to `firstValue`, the last value is equal to `lastValue`, and the other values are evenly distributed between these two limits. Returns `true` if it successfully created the array, `false` otherwise.
2. `bool minutesToString(double timeInMinutes, char outputString[LENGTH])`
   Calculates and returns a one-dimensional character array (i.e., a string) representing the given number of fractional minutes in the human readable format `HH:MM:SS`, where `HH` is the integer number of hours, `MM` is the integer number of minutes, and `SS` is the integer number of seconds, truncating any fractional seconds. Returns `true` if it successfully created the string, `false` otherwise.
3. `bool numericalPaceTable(double distances[COLUMNS], double paces[ROWS], double outputArray[ROWS][COLUMNS])`
   Calculates and returns a two dimensional array containing the number of minutes it takes to run the given distances, `distances[COLUMNS]`, at the given paces, `paces[ROWS]`. Each element of the resulting array, `outputArray[i][j]`, is the time required to run the distance given as `distances[j]` at the running pace given as `paces[i]`. Returns `true` if it successfully created the array, `false` otherwise. This function can be called using results created with `linespace`.
4. `stringPaceTable(double distances[COLUMNS], double paces[ROWS], char outputArray[ROWS][COLUMNS][LENGTH])`
   Calculates and returns a three dimensional array of characters (i.e., a two-dimensional array of strings) containing the time it takes to run the given distances, `distances[COLUMNS]`, at the given paces, `paces[ROWS]`, as a human-readable string of the format `HH:MM:SS`, where `HH` is the integer number of hours, `MM` is the integer number of minutes, and `SS` is the integer number of seconds, truncating any fractional seconds. Each element of the

 ©2026 Queen's University

resulting array, `outputArray[i][j]`, is the time required to run the distance given as `distances[j]` at the running pace given as `paces[i]`. Returns `true` if it successfully created the array, `false` otherwise. This function will need to use `minutesToString`. This function can be called using results created with `linespace`.

You will be assessed on the correctness of the output of these four functions. You may choose to implement their internal details however you wish, but **they must have the function signatures (e.g., names, arguments, return values) listed above, including the compiler macros `COLUMNS`, `ROWS`, `LENGTH`**. To ensure that the compiler macros are defined correctly, a basic starter file is provided on OnQ.

To receive full marks you should use memory safe techniques when working with arrays and strings as discussed in class and labs.

You will likely find it helpful to also write a `main` function that you can use to test your functions against the example output provided in Table 2. We will not assess your `main` function.

Comments are mandatory for this assignment. Add comments as necessary for key pieces in your code to explain what the program is doing, including any function definitions, assumptions, variable declaration, conditional statements, looping conditions, etc.

The results of your functions must match the sample output (Table 2); otherwise, the auto grading software will not be able to grade your assignment, which may affect your mark.

2.1 Assumptions

You can make the following assumptions when writing your functions:

1. `bool linspace(double firstValue, double lastValue, int numValues, double outputArray[])`
   The array, `outputArray`, will have at least as many elements as the given `numValues`. The function can only calculate the array if `numValues` is greater or equal to 2, i.e., `numValues` ≥ 2.

2. `bool minutesToString(double timeInMinutes, char outputString[LENGTH])`
   The time in minutes is converted to integer numbers of hours, minutes, and seconds (i.e., fractional seconds are truncated). The number of hours will never be greater than 99.

3. `bool numericalPaceTable(double distances[COLUMNS], double paces[ROWS], double outputArray[ROWS][COLUMNS])`
   The distances are given in kilometres and the paces are given in minutes per kilometres.

4. `stringPaceTable(double distances[COLUMNS], double paces[ROWS], char outputArray[ROWS][COLUMNS][LENGTH])`
   The distances are given in kilometres and the paces are given in minutes per kilometres. The time is converted to integer numbers of hours, minutes, and seconds (i.e., fractional seconds are truncated). The number of hours will never be greater than 99.

 ©2026 Queen's University

2.2 Existing Functions

Like most programming languages, there are a number of existing, useful functions you can use in C. These functions are no different than the ones you will write yourself, other than that they have been rigorously tested and made publicly available to everyone!

Functions provided by `string.h` and `stdio.h` that you may want to use for this assignment include:

- `int strnlen(char str[], int num)`
  Returns the length of the string, `str`, which is known to be no larger than `num`.
- `char* strncat(char destination[], char source[], int num)`
  Appends no more than `num` characters of the string `source` onto the end of the string `destination`.
- `int snprintf(char str[], int num, char format[], ... )`
  Creates a string using the format string, `format`, and any required variables and stores the first `num` characters of it in the string `str`.

You can read more about these functions, `string.h`, and `stdio.h` at https://cplusplus.com/reference/cstring/ and https://cplusplus.com/reference/cstdio/.

3 Background

A pace table is a calculation of how long it would take to run different distances at various running paces, which are defined as the time to run one kilometre. For example, the pace for the 5 kilometre world-record is just under 2.52 minutes per kilometre (i.e., 2 minutes and 31 seconds, or 00:02:31) which means the record time is just under 12.6 minutes (i.e., 12 minutes and 36 seconds, or 00:12:36).

Pace tables are presented with each running pace corresponding to a row and each distance corresponding to a column. The value in each cell is the time required to run the distance at the pace, e.g., the time to run 5 kilometres at a pace of 5.5 minutes per kilometre is 27.5 minutes. An example pace table for the paces 6.0, 5.5, 5.0, 4.5, and 4.0 minutes per kilometre and the running distances 1, 5, 10, 21.1 and 42.2 kilometres is presented as Table 1.

This assignment requires you to create functions to calculate the body of a pace table in both fractional minutes (e.g., 27.5 minutes) and strings of integer hours, minutes, and second (e.g., 00:27:30). This includes functions to create an evenly space array of values, convert from a fractional number of minutes to a hour, minute and second string, and calculate the bodies of the functions themselves.

 ©2026 Queen's University

Table 1: Example pace table with headers

	Distances [km]				
Pace [min/km]	**1.0**	**5.0**	**10.0**	**21.1**	**42.2**
6.0	00:06:00	00:30:00	01:00:00	02:06:36	04:13:12
5.5	00:05:30	00:27:30	00:55:00	01:56:03	03:52:06
5.0	00:05:00	00:25:00	00:50:00	01:45:30	03:31:00
4.5	00:04:30	00:22:30	00:45:00	01:34:57	03:09:54
4.0	00:04:00	00:20:00	00:40:00	01:24:24	02:48:48

3.1 Guidance and Suggestions

You may work on this assignment however you wish, but below is a guideline on how you may wish to approach this assignment:

1. Download the provided `apsc143assign3.c` file which has the required compiler macros definitions.
2. Create signatures for the required functions and an `int main(void)` function.
3. Write the details of the `linspace` function.
4. Call `linspace` in your `main` and print its output to confirm the results with Table 2a.
5. Repeat steps 3 & 4 separately for `minutesToString`, `numericalPaceTable`, and `stringPaceTable`. You can use `linespace` to create test inputs to `numericalPaceTable` and `stringPaceTable`.

Some additional hints that may help you complete the assignment:

- A three-dimensional character array can be thought of as a two-dimensional array of strings, e.g.,

  ```
  char stringArray[ROWS][COLUMNS][LENGTH] = {'\0'};
  ```

  can be viewed as creating a two-dimensional array of size `ROWS`×`COLUMNS` where each element of this two-dimensional array is a string up to `LENGTH` characters long and each string is initialized to the empty string, `""`. We can use this variable as a two dimensional array of strings by ignoring the last set of indices, e.g.,

  ```
  snprintf(stringArray[0][1], LENGTH, "My string");
  ```

  will put the string “My string” into the the array at position `[0][1]` after which our array would contain the strings:

  ```
  stringArray[0][0] → ""
  stringArray[0][1] → "My string"
  stringArray[1][0] → ""
  ```
  etc.

 ©2026 Queen’s University

Table 2: Expected Results

(a) Expected outputs of `linspace(firstValue, lastValue, numValues, outputArray)`

`firstValue`	`lastValue`	`numValues`	`outputArray`
4	6	2	{4.00, 6.00}
4	6	5	{4.00, 4.50, 5.00, 5.50, 6.00}
3.5	4.5	11	{3.50, 3.60, 3.70, 3.80, 3.90, 4.00, 4.10, 4.20, 4.30, 4.40, 4.50}
7	6	4	{7.00, 6.67, 6.33, 6.00}

(b) Expected outputs of `minutesToString(timeInMinutes, outputString)`

`timeInMinutes`	`outputString`
90.00	01:30:00
36.80	00:36:48
15.33	00:15:19

(c) Expected outputs of `numericalPaceTable(distances, paces, outputArray)` with `distances = {1.00, 2.00, 3.00, 4.00, 5.00, 6.00, 7.00, 8.00}` and `paces = {4.00, 4.50, 5.00, 5.50, 6.00}`.

4.00	8.00	12.00	16.00	20.00	24.00	28.00	32.00
4.50	9.00	13.50	18.00	22.50	27.00	31.50	36.00
5.00	10.00	15.00	20.00	25.00	30.00	35.00	40.00
5.50	11.00	16.50	22.00	27.50	33.00	38.50	44.00
6.00	12.00	18.00	24.00	30.00	36.00	42.00	48.00

(d) Expected outputs of `numericalPaceTable(distances, paces, outputArray)` with `distances = {10.00, 12.00, 14.00, 16.00, 18.00, 20.00, 22.00, 24.00}` and `paces = {8.00, 7.75, 7.50, 7.25, 7.00}`

80.00	96.00	112.00	128.00	144.00	160.00	176.00	192.00
77.50	93.00	108.50	124.00	139.50	155.00	170.50	186.00
75.00	90.00	105.00	120.00	135.00	150.00	165.00	180.00
72.50	87.00	101.50	116.00	130.50	145.00	159.50	174.00
70.00	84.00	98.00	112.00	126.00	140.00	154.00	168.00

(e) Expected outputs of `stringPaceTable(distances, paces, outputArray)` with `distances = {1.00, 2.00, 3.00, 4.00, 5.00, 6.00, 7.00, 8.00}` and `paces = {4.00, 4.50, 5.00, 5.50, 6.00}`

00:04:00	00:08:00	00:12:00	00:16:00	00:20:00	00:24:00	00:28:00	00:32:00
00:04:30	00:09:00	00:13:30	00:18:00	00:22:30	00:27:00	00:31:30	00:36:00
00:05:00	00:10:00	00:15:00	00:20:00	00:25:00	00:30:00	00:35:00	00:40:00
00:05:30	00:11:00	00:16:30	00:22:00	00:27:30	00:33:00	00:38:30	00:44:00
00:06:00	00:12:00	00:18:00	00:24:00	00:30:00	00:36:00	00:42:00	00:48:00

 ©2026 Queen's University

(f) Expected outputs of `stringPaceTable(distances, paces, outputArray)` with `distances = {10.00, 12.00, 14.00, 16.00, 18.00, 20.00, 22.00, 24.00}` and `paces = {8.00, 7.75, 7.50, 7.25, 7.00}`

01:20:00	01:36:00	01:52:00	02:08:00	02:24:00	02:40:00	02:56:00	03:12:00
01:17:30	01:33:00	01:48:30	02:04:00	02:19:30	02:35:00	02:50:30	03:06:00
01:15:00	01:30:00	01:45:00	02:00:00	02:15:00	02:30:00	02:45:00	03:00:00
01:12:30	01:27:00	01:41:30	01:56:00	02:10:30	02:25:00	02:39:30	02:54:00
01:10:00	01:24:00	01:38:00	01:52:00	02:06:00	02:20:00	02:34:00	02:48:00

4 Submission Instructions

Create your program and upload it to Gradescope for grading. Your program file **must be named** `apsc143assign3.c` in order for your assignment to be graded. Do not include any personal information (student number, name, etc.) in your submission.

Include a comment in your code attesting to the originality of your work and that you have not used Generative AI.

Refer to the assignment rubric below for a detailed breakdown of the grading criteria. Your submission must adhere to the assignment rules as outlined in the submission policy document for this course, which can also be found on OnQ.

There is zero tolerance for plagiarism in this course. The auto grading software will automatically flag potential cases of plagiarism, which will be reviewed by the teaching team.

More information on assignment submissions can be found on OnQ, and information on the specific definition and repercussions of plagiarism can be found in the "Begin Here (About This Course)" module.

4.1 Assignment Rubric

Test case marks are based purely on code output when given specific inputs. Code design, style, and commenting are graded by the teaching team looking at your code. Use comments effectively to not only get full marks in the comments section, but to also make your code clearer to the marking TA.

A portion of each assignment's marks will be given for your understanding of your code. If your code only uses the concepts taught so far in the course, then you will receive these marks automatically. If your code uses other concepts, including those not yet presented or not covered in the course, then in order to receive these marks you may be asked to explain to a member of the teaching team how your code works and why you chose to solve the problem that way.

 ©2026 Queen's University

Table 3: Marking Rubric — Output

Test Cases (Marks will only be awarded if the code compiles)	**Level 3 (3 points)**	**Level 2 (2 points)**	**Level 1 (1 points)**	**N/A (0 points)**	**Criterion Score**
Assignment Correctness - `linspace`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `minutesToString`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `numericalPaceTable`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - `stringPaceTable`	Actual output perfectly matches expected output.	Some of the required values are correct.	Few output values are correct.	The code did not compile or crashed during runtime.	/3
Assignment Correctness - Hidden Test Cases	Actual output perfectly matches expected output.	Some of the required values are correct.	Few values are correct.	The code did not compile or crashed during runtime.	/3

©2026 Queen's University

Table 4: Marking Rubric — Style

Code Style	**Level 2 (2 points)**	**Level 1 (1 points)**	**Level 0 (0 points)**	**Criterion Score**
Memory Safety	Code always follows best practices for memory safety, e.g., compiler macros for array sizes and fixed-length versions of string functions.	Code occasionally does not follow best practices for memory safety.	Code does not follow best practices for memory safety.	/2
Coding Style	The coding style is consistent, clear, and well organized.	1 or more instances exist where coding style is not considered.	The coding style lacks clarity.	/2
Commenting	Code is well documented. Comments provide clarity for all tasks and logic.	Few comments are used, major tasks are undocumented and logic is difficult to follow. Comments included are vague and ineffective.	No meaningful comments are used.	/2

©2026 Queen's University

Table 5: Marking Rubric — Understanding

Code Understanding (Marks will only be awarded if the code compiles)	**Level 3 (9 points)**	**Level 2 (6 points)**	**Level 1 (3 points)**	**Level 0 (0 points)**	**Criterion Score**
Code Understanding	Student uses concepts covered in class that are assumed to be understood or uses concepts not covered in the class and can clearly explain and justify their submitted code to a member of the teaching team.	Student uses concepts not covered in class and can explain their submitted code to a member of the teaching team. Their justification of aspects of their submitted code to a member of the teaching team lacks clarity.	Student uses concepts not covered in class. Their explanation and justification of aspects of their submitted code to a member of the teaching team lacks clarity.	Student uses concepts not covered in class and cannot explain nor justify aspects of their submitted code to a member of the teaching team.	/9

©2026 Queen's University